

EvoCracker

AI RESEARCH PROJECT

Phase 3 of 5 — The Code

PHASE 3

The Code

Search algorithms, real-time behavior management & the Genetic Algorithm loop

Deliverable Summary

- Implemented all required search algorithms from scratch.
- Wired runtime behavior-state changes to path requests and movement updates.
- Completed a full GA lifecycle (evaluation, selection, crossover, mutation, replacement).

Verification Checklist

- ✓ Pathfinding outputs include comparability metrics.
- ✓ Runtime AI state flow is documented end-to-end.
- ✓ GA operations and inputs are documented in production terms.

1. Pathfinding Implementation

Pathfinding is implemented in `src/ai/pathfinding/` and routed through `AlgorithmRegistry.ts`.

Algorithm	Description
BFS	Complete, unweighted breadth-first search.
DFS	Depth-first traversal for exploratory/non-optimal pursuit.

IDS	Iterative deepening search.
DLS	Depth-limited search with genome-driven depth limits.
UCS	Uniform-cost search using tile movement weights.
AStar	Heuristic pathfinding using g + h.
GreedyBFS	Heuristic-only best-first search.
HillClimbing	Local heuristic improvement for fast but incomplete movement.

Algorithm Return Values

Each algorithm returns a standardized result object:

Field	Description
path	Selected route as an array of grid coordinates
nodesExpanded	Search effort — nodes actively processed
nodesVisited	Total count of nodes visited during search
timeMs	Runtime in milliseconds for comparative analysis
success	Boolean — whether a valid path was found
expansionOrder	Visualization data for the algorithm lab/dashboard

2. Real-Time State Management

Enemies are controlled by EnemyBase.ts and the Behavior Tree engine in src/ai/behavior/BehaviorTree.ts.

- 1

VisionSystem.ts updates whether the player is visible using raycast line-of-sight.
- 2

Enemy blackboards store player position, last known position, alert timers, health, and path state.
- 3

The Behavior Tree transitions the enemy between idle, suspicious, alert/chasing, and fleeing states.
- 4

EnemyBase.requestPath() asks the pathfinding client for a path to the current behavior target.
- 5

EnemyBase.update() follows the path, attacks on contact, tracks survival time, and logs search metrics.

3. Genetic Algorithm Loop

The Genetic Algorithm is implemented in `src/ai/evolution/GeneticAlgorithm.ts`.

Genome Genes

Gene / Trait	Description
<code>speed</code>	Movement velocity multiplier.
<code>vision</code>	FOV range and arc modifier.
<code>aggression</code>	Attack vs. flee tendency.
<code>persistence</code>	Chase duration after losing line of sight.
<code>cautiousness</code>	Hesitation before engaging or entering unknown areas.
<code>packTendency</code>	Desire to group with other agents.
<code>ambushTendency</code>	Preference for choke-point hiding.
<code>patrolVariance</code>	Randomness in patrol route selection.
<code>algorithmWeights</code>	Weighted preference across all 8 search algorithms.

Evolution Process

- 1

Enemy performance is measured during a run.
- 2

`calculateFitness()` scores enemies using visibility, damage, detections, survival, area coverage, stuck time, and cooperative pressure.
- 3

`tournamentSelection()` or `rouletteSelection()` chooses parents.
- 4

`uniformCrossover()` or `weightedAverageCrossover()` creates children.
- 5

`mutateGenome()` applies Gaussian mutation to traits and algorithm weights.
- 6

`applyPlaystyleBias()` nudges the population based on the classified player style.
- 7

`computeGenerationStats()` produces dashboard metrics.

4. Algorithm Diversity

Design

Enemy pursuit uses weighted sampling from `algorithmWeights` instead of always choosing the highest weight. This keeps evolution meaningful while preventing every enemy from collapsing into the same algorithm after a few generations.

The strongest algorithms still become more likely, but weaker algorithms remain observable for analysis and comparison in the Pathfinding dashboard.